

Explicación del Código Backend: RENIEC Sechura

1. Capa de Datos (Entidades de Dominio)

Ubicación: `src/main/java/reniec/sechura/domain/`

Estas clases representan las tablas de la base de datos. Utilizamos las anotaciones de ****JPA (Java Persistence API)**** para decirle a Spring Boot cómo convertir el código Java en tablas de MySQL.

Herencia: Persona y Usuario

* `Persona.java` : Es la clase padre para datos civiles. Usamos `@Inheritance(strategy = InheritanceType.JOINED)` . Esto significa que creará una tabla `persona` , y cuando guardemos un `Pescador` , creará un registro en la tabla `pescador` enlazado al ID de la persona.

* `Usuario.java` : Es la clase padre para el acceso al sistema. Contiene el `username` y `passwordHash` .

* `Ciudadano.java` : Hereda de `Usuario` (para poder iniciar sesión) y se relaciona mediante `@OneToOne` con `Persona` (para tener los datos civiles).

Entidades del Flujo de Negocio

* `Tramite.java` : El núcleo de las gestiones. Se relaciona con `Ciudadano` , `TipoTramite` , y `EstadoTramite` . Usamos `@PrePersist` para que, justo antes de guardar en la base de datos por primera vez, se asigne automáticamente la `fechaSolicitud` con la fecha actual.

* `Cita.java` : Relaciona un `Tramite` con un `Horario` . Contiene la confirmación final y el "Ticket Digital".

* `Horario.java` : Define la capacidad y cuántas citas actuales existen.

* `EstadoTramite.java` : Un `enum` (enumeración) que lista los estados permitidos: `PENDIENTE` , `EN\_PROCESO` , etc., garantizando que no se puedan ingresar estados inválidos.

2. Capa de Datos (Repositorios)

Ubicación: `src/main/java/reniec/sechura/repository/`

Los repositorios son las interfaces que realizan las consultas a la base de datos (SELECT, INSERT, UPDATE, DELETE) sin que nosotros escribamos código SQL directo.

*****Ejemplo: `TramiteRepository.java`*****

```
```java
public interface TramiteRepository extends JpaRepository<Tramite, Integer> {
 Optional<Tramite> findByCodigoTramite(String codigoTramite);
 List<Tramite> findByCiudadanoIdUsuario(Integer idUsuario);
}
```
```

Al extender `JpaRepository`, Spring Boot automáticamente nos regala métodos como `.save()`, `.findAll()`, `.findById()`, etc.

Además, al declarar `findByCiudadanoIdUsuario`, Spring Boot es lo suficientemente inteligente para entender que queremos "Buscar todos los trámites filtrando por el ID de Usuario del Ciudadano", escribiendo la consulta SQL por detrás de forma automática.

3. Capa de Negocio (Servicios)

Ubicación: `src/main/java/reniec/sechura/service/`

Aquí es donde reside la "inteligencia" o lógica de tu sistema. Los servicios validan reglas antes de tocar la base de datos. Se marcan con la anotación `@Service`.

*****`UsuarioService.java`***:**

*** **`autenticarPorDni(String dni)`**:** Recibe el DNI del usuario, primero busca en nuestra base de datos si ya está registrado. Si no está, hace una petición HTTP automática a la API de ****perudevs.com**** (usando tu token privado) mediante ``RestTemplate``. Extrae los nombres y apellidos de la respuesta, crea el registro en nuestra BD local automáticamente y permite el acceso. Además, usamos ``@SuppressWarnings("unchecked")`` para mantener el código limpio de advertencias del compilador. ¡Esto mejora enormemente la experiencia de usuario (UX)!

*** **`TramiteService.java`**:**

*** **`crearTramite(...)`**:** Aquí se genera dinámicamente un código único para el trámite usando ``UUID.randomUUID()`` para que tenga el formato solicitado en el documento técnico (ej. ``TRM-ABCD1234``). Luego fija el estado en ``PENDIENTE`` y lo guarda.

*** **`CitaService.java`**:**

*** **`reservarCita(...)`**:** Contiene la lógica central de validación.

1. Busca el Trámite y el Horario.
2. Comprueba si el Horario tiene cupo disponible (``citasActuales < capacidadMax``).
3. Comprueba si el Trámite ya tiene una cita (para evitar duplicados).
4. Si todo es válido, incrementa ``citasActuales`` en +1. Si llega al límite, bloquea el horario poniendo ``disponible = false``.
5. Finalmente, crea la Cita y le asigna un Ticket Digital (``QR-XYZ``).

4. Capa de Vista / API (Controladores)

Ubicación: ``src/main/java/reniec/sechura/controller/``

Son los puentes de comunicación. Exponen URLs (endpoints) a través de la web para que una aplicación móvil, frontend (React/Angular) o Postman puedan interactuar con el sistema. Se usa ``@RestController``.

*** ** `AuthController.java` **:**

* Maneja la petición `@PostMapping("/api/auth/login")`. Ahora solo pide el `dni` como parámetro. Llama a `UsuarioService` para validar contra la API y devuelve el `Ciudadano` registrado.

*** ** `TramiteController.java` **:**

* Maneja las peticiones relacionadas a trámites en la ruta `/api/tramites`.

* `crearTramite` espera un JSON con los datos del trámite (`@RequestBody`) y se lo envía al servicio para ser procesado.

*** ** `CitaController.java` **:**

* `@GetMapping("/api/citas/horarios/{idOficina}")` : Expone una url donde, al pasarle el ID de la oficina, retorna en formato JSON la lista de todos los horarios disponibles.

5. Configuración

Ubicación: `src/main/resources/application.properties`

Es el archivo de propiedades del proyecto.

* `spring.datasource.url=jdbc:mysql://localhost:3306/reniec\_segura...` -> Le dice a Spring que se conecte al motor MySQL local y que cree la base de datos `reniec\_segura` si no existe.

* `spring.jpa.hibernate.ddl-auto=update` -> Es la instrucción clave que hace que Spring Boot analice nuestras Entidades (Capa de datos) y ****cree o actualice automáticamente las tablas y columnas**** en MySQL cada vez que corramos el proyecto.